

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



Môn: THỰC TẬP CƠ SỞ
BÁO CÁO BÀI TẬP LỚN

“Xây dựng hệ thống sửa lỗi chính tả cho tiếng Việt”

Sinh viên thực hiện: Phạm Quang Anh

Mã sinh viên: B23DCCN044

Lớp: D23CQCN02-B

Nhóm: 11

Giảng viên hướng dẫn: TS.Kim Ngọc Bách

Hà Nội 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành tới Học viện Công nghệ Bưu chính Viễn thông vì đã tạo điều kiện cho chúng em được tham gia môn học Thực tập cơ sở. Đây là một môn học rất bổ ích và thiết thực, trang bị cho chúng em những kiến thức nền tảng quan trọng trong lĩnh vực Công nghệ thông tin.

Chúng em đặc biệt biết ơn thầy Kim Ngọc Bách, người đã tận tâm dẫn dắt chúng em trong suốt hành trình chinh phục môn học này. Thầy không chỉ truyền đạt những kiến thức chuyên sâu một cách dễ hiểu mà còn nhiệt tình giải đáp mọi thắc mắc, chia sẻ những kinh nghiệm thực tế quý báu, giúp chúng em có cái nhìn toàn diện và sâu sắc hơn.

Quá trình hoàn thành báo cáo bài tập lớn là một thử thách không nhỏ đối với chúng em. Tuy thời gian có hạn và gặp phải không ít khó khăn, nhưng với sự hướng dẫn tận tình của thầy và tinh thần học hỏi, nỗ lực không ngừng, chúng em đã cố gắng hết sức để hoàn thành báo cáo này một cách tốt nhất.

Kính mong thầy đánh giá và góp ý để chúng em có thể rút kinh nghiệm, hoàn thiện bản thân và phát triển hơn nữa trên con đường học tập sắp tới.

MỤC LỤC

LỜI CẢM ƠN

1.1 Lý do chọn đề tài

1.2 Mục tiêu cần đạt được

1.3. Tính ứng dụng

1.4. Giá trị học thuật

2.1 Kiến trúc Transformer (Encoder-Decoder)

2.1.1 Thành phần mã hóa (Encoder Stack)

2.2 Mô hình Pre-trained BARTpho

2.2.1 Kiến trúc của mô hình BERT và GPT

2.2.2 Mô hình BART

2.2.3 Mô hình BARTPho

2.3 Gradio Framework

2.3.1 Kiến trúc Client-Server và Cơ chế Giao tiếp Bất đồng bộ

2.3.2 Cơ chế Đồng bộ Hàng đợi và Quản lý Luồng (Concurrency & Queueing)

2.3.3 Phân tích Xác suất Softmax và Kết xuất Độ tự tin (Confidence Scoring)

3.1 Yêu cầu hệ thống và đặc tả chức năng

3.1.1 Yêu cầu chức năng

3.1.2 Yêu cầu phi chức năng

3.2 Thiết kế kiến trúc tổng thể End-to-End

4.1 Môi trường cài đặt và triển khai hệ thống

4.1.1 Môi trường huấn luyện

4.1.2 Môi trường kiểm thử trực quan

4.1.3 Hệ sinh thái Học máy Hugging Face

4.1.4 Nền tảng điện toán đám mây và kho dữ liệu Kaggle

4.2 Triển khai mã nguồn ứng dụng thực tế

1. Clone repository

2. Cài đặt thư viện

3. Chạy ứng dụng giao diện chính (Gradio Web App):

4.3 Phân tích động lực học huấn luyện qua đồ thị TensorBoard

4.3.1 Phân tích động lực học huấn luyện và độ hội tụ mô hình

4.3.2 Về hiệu năng xử lý phần cứng và thời gian thực

4.3.3 Đánh giá định lượng qua thước đo tiêu chuẩn WER và CER

4.4 Đánh giá kiểm thử trực quan trên giao diện ứng dụng (Gradio Web UI)

5.1 Kết quả đạt được

5.2 Hạn chế và Hướng phát triển đề tài

1. Giới thiệu chung

1.1 Lý do chọn đề tài

Sửa lỗi chính tả tiếng Việt là yếu tố quan trọng để nâng cao chất lượng các hệ thống xử lý dữ liệu dựa trên văn bản. Sửa lỗi chính tả cho tiếng Việt góp một phần lớn trong công việc tiền xử lý dữ liệu, tinh chỉnh và cải thiện kết quả từ tác vụ OCR nhằm số hóa tài liệu

1.2 Mục tiêu cần đạt được

Dự án tập trung vào việc nghiên cứu và triển khai hệ thống tự động hóa quá trình kiểm tra và sửa lỗi văn bản, hướng tới các mục tiêu cụ thể sau:

- **Xây dựng mô hình lỗi hiệu quả:** Phát triển hệ thống dựa trên kiến trúc Transformer hoặc Sequence-to-Sequence (Seq2Seq) để hiểu sâu ngữ cảnh của câu.
- **Độ chính xác cao:** Đạt tỷ lệ sửa lỗi chính xác ít nhất **85%** trên các tập dữ liệu chuẩn thông qua việc tinh chỉnh (fine-tuning) các mô hình tiền huấn luyện như ViT5 hoặc BARTpho.
- **Xử lý đa dạng loại lỗi:**
 - **Lỗi gõ máy:** Khắc phục các lỗi sai ký tự, thiếu dấu hoặc sai quy tắc gõ Telex/VNI.
 - **Lỗi ngữ pháp:** Gợi ý sửa các từ dùng sai ngữ cảnh dựa trên xác suất ngôn ngữ.
 - **Lỗi OCR:** Đặc biệt tối ưu hóa khả năng nhận diện và sửa các lỗi do quét văn bản (OCR) như nhầm lẫn giữa các ký tự có hình dáng tương đồng (0-o, l-i, q-g).

- **Khả năng mở rộng:** Triển khai hệ thống bằng thư viện Hugging Face Transformers hoặc PyTorch để đảm bảo tính linh hoạt và dễ dàng tích hợp vào các ứng dụng thực tế.

1.3. Tính ứng dụng

Hệ thống sửa lỗi chính tả có khả năng triển khai thực tế trong nhiều lĩnh vực:

- **Chuẩn hóa văn bản hành chính:** Hỗ trợ soạn thảo văn bản, giảm thiểu sai sót trong các giấy tờ quan trọng của tổ chức và doanh nghiệp.
- **Tối ưu hóa số hóa tài liệu (OCR):** Tự động chỉnh sửa các lỗi phát sinh sau khi quét tài liệu cũ hoặc văn bản viết tay, giúp quá trình lưu trữ số đạt độ chính xác cao.
- **Hỗ trợ biên tập nội dung:** Tích hợp vào các nền tảng báo chí, blog và mạng xã hội để đảm bảo tính chuyên nghiệp của nội dung trước khi xuất bản.
- **Cải thiện trải nghiệm người dùng:** Tích hợp vào bộ gõ trên thiết bị di động để gợi ý từ đúng ngay khi người dùng nhập liệu sai.

1.4. Giá trị học thuật

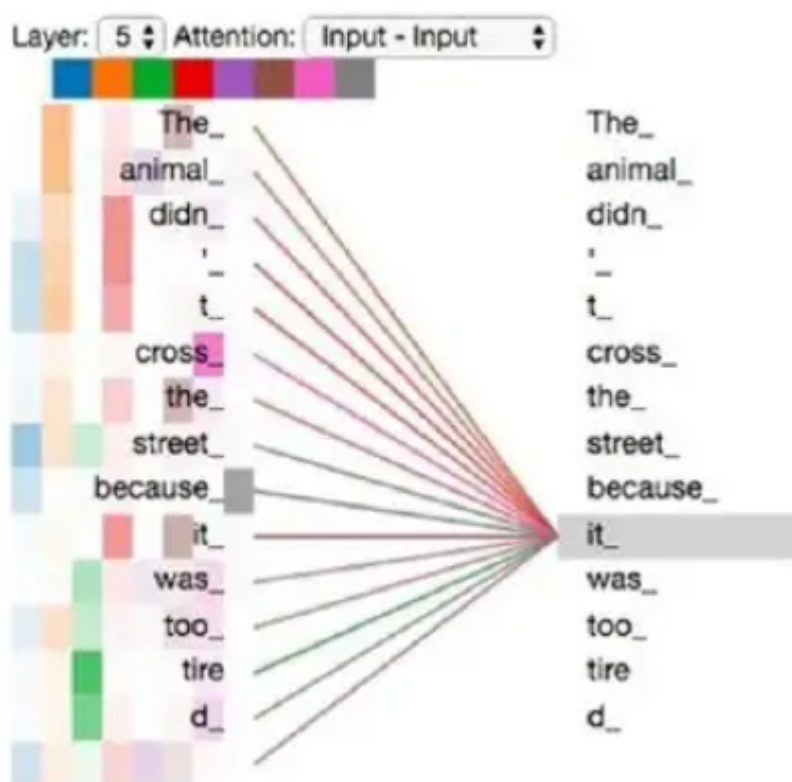
Đề tài ứng dụng mô hình **BARTpho-syllable** – kiến trúc dựa trên Transformer được đào tạo trước (Pre-trained) với quy mô dữ liệu tiếng Việt khổng lồ. Thay vì sửa lỗi theo bộ quy tắc cứng hoặc từ điển tĩnh vốn dễ bỏ sót lỗi ngữ cảnh, nghiên cứu này đóng góp một giải pháp dựa trên **Ngữ cảnh ngữ nghĩa sâu (Deep Semantic Context)**. Mô hình có khả năng tự "hiểu" mối liên kết giữa các từ xung quanh để đưa ra dự đoán chính xác nhất cho từ bị lỗi, mở ra hướng tiếp cận hiện đại cho các bài toán Seq2Seq tiếng Việt.

2. Cơ sở lý thuyết và công nghệ sử dụng

Trong phạm vi nghiên cứu và thực nghiệm của đề tài, hệ thống lựa chọn ứng dụng phiên bản **BARTpho-syllable** (phân tích ở mức độ âm tiết) thay vì phiên bản BARTpho-word. Lựa chọn này dựa trên đặc thù

của bài toán sửa lỗi chính tả và khôi phục dấu tiếng Việt, nơi dữ liệu đầu vào thường xuyên xuất hiện các biến thể cấu trúc từ phức tạp, từ viết tắt, teencode hoặc lỗi dính chữ do quá trình quét văn bản (OCR). Việc phân tích dữ liệu ở cấp độ âm tiết giúp kiến trúc Transformer xử lý linh hoạt chuỗi ký tự nhiễu, tránh hiện tượng bùng nổ token không xác định và tối ưu hóa năng lực dịch dịch chuỗi Seq2Seq từ câu lỗi sang câu chuẩn ngữ nghĩa.

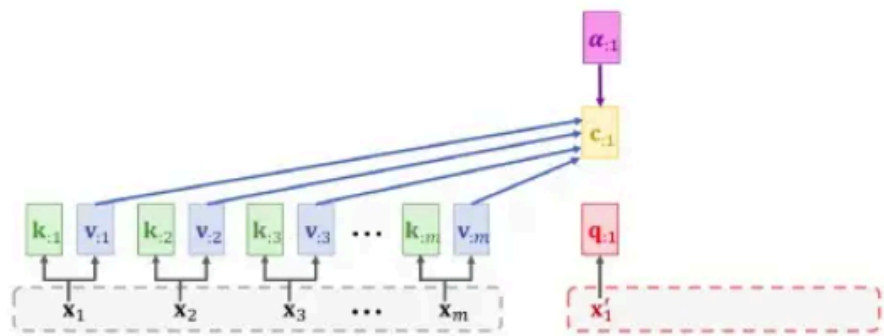
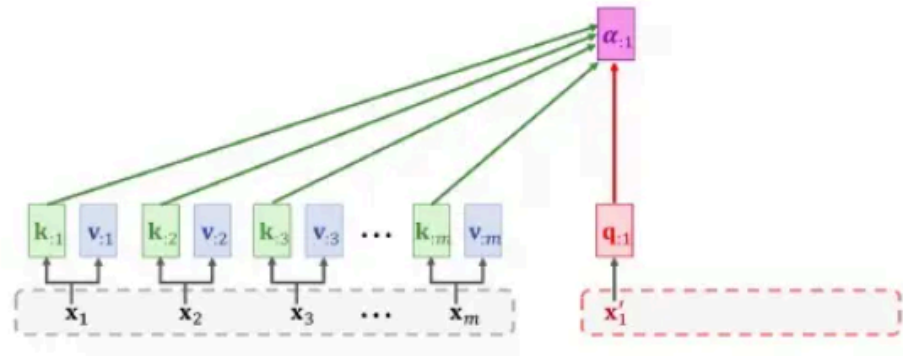
2.1 Kiến trúc Transformer (Encoder-Decoder)



2.1.1 Thành phần mã hóa (Encoder Stack)

Encoder nhận đầu vào là chuỗi xung đột mã hóa từ vựng (Tokenized Tokens) dính lỗi hoặc nhiễu cấu trúc ký tự. Nhiệm vụ chính của Encoder là ánh xạ chuỗi này vào không gian vector liên tục biểu diễn ngữ cảnh hai chiều (Bi-directional Contextual Representation).

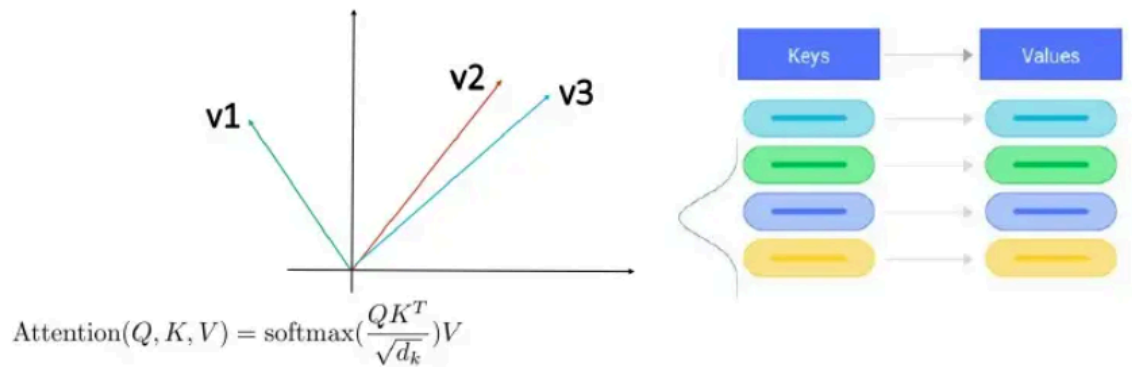
- **Cơ chế Chú ý Đầu nhiều nhánh (Multi-Head Self-Attention):**



Tại mỗi lớp mã hóa, các vector trạng thái ẩn được chiếu tuyến tính thành ba ma trận: Truy vấn (Q), Khóa (K), và Giá trị (V). Hàm tính toán điểm chú ý được định nghĩa qua công thức Scaled Dot-Product Attention:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó d_k là chiều không gian của khóa. Cơ chế Multi-Head cho phép mô hình đồng thời quan sát các mối quan hệ ngữ nghĩa ở cả tầm xa và tầm gần (ví dụ: liên kết từ dính liền phía trước với từ sai chính tả phía sau).



Như hình trên, khi hai vector được **dot product** với nhau. Nếu hai vector gần nhau thì kết quả sẽ ra giá trị rất lớn, nếu hai vector ngược hướng thì giá trị sẽ bị triệt tiêu. Khi đó kết quả phân phối từ phép tính dot product của Key và Value sẽ làm **nổi bật và tập trung** vào những vector có cùng hướng hay sự tương đồng với nhau. Còn những vector ngược hướng thì kết quả sẽ rất nhỏ hoặc độ lớn không đáng kể. Sau quá trình chuẩn hóa sẽ nhân với ma trận value. Kết quả cuối cùng chúng ta sẽ tính được sự tương quan của một vector trong encoder đối với tất cả các vector khác trong lớp decoder

- **Mạng nơ-ron tiến một hướng (Position-Wise Feed-Forward Network - FFN):** Gồm hai phép biến đổi tuyến tính kết hợp hàm kích hoạt phi tuyến tính nhằm tinh chỉnh các đặc trưng sau khi được tổng hợp qua tầng chú ý:

$$FFN(x) = \left(0, xW_1 + b_1\right) W_2 + b_2$$

2.1.2 Thành phần giải mã (Decoder Stack)

Decoder chịu trách nhiệm sinh tự hồi quy (Auto-regressive) chuỗi ký tự sạch đích, tức là dự đoán từ tiếp theo dựa trên các từ tổ đã sinh ra trước đó và ma trận ngữ cảnh nhận về từ Encoder.

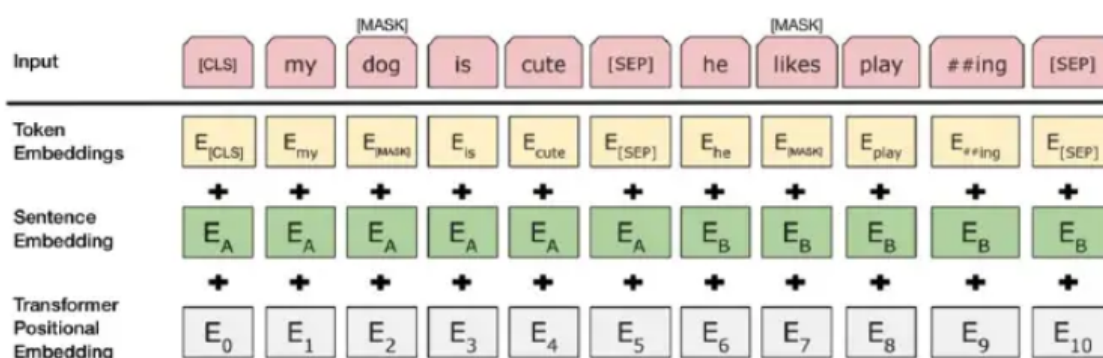
- **Masked Multi-Head Attention:** Để đảm bảo tính phân tách thời gian trong chế độ tự hồi quy, tầng tự chú ý của Decoder áp dụng một ma trận mặt nạ (Masking). Cơ chế này đặt các trọng số chú ý của các từ tổ tương lai về $-\infty$ trước khi qua hàm

softmax, ngăn chặn việc mô hình "nhìn trước" đáp ứng đích trong quá trình huấn luyện.

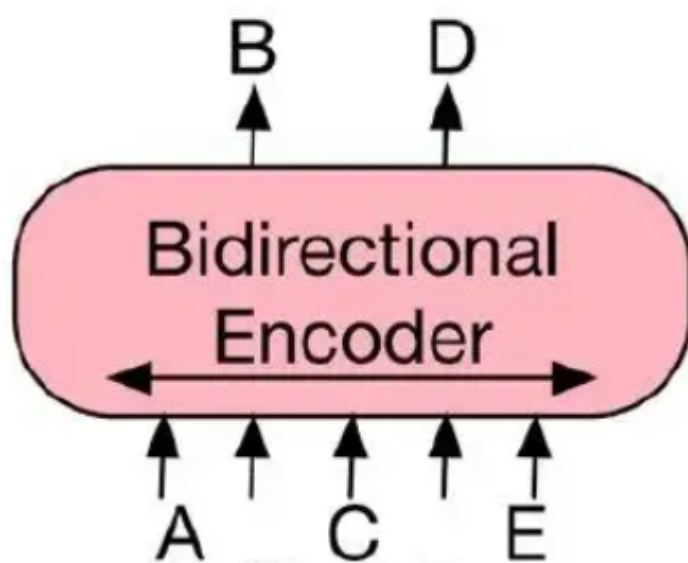
- **Encoder-Decoder Cross-Attention:** Đây là cầu nối thông tin quan trọng nhất. Ma trận truy vấn Q được trích xuất từ trạng thái ẩn phía trước của Decoder, trong khi ma trận khóa K và giá trị V được lấy trực tiếp từ đầu ra cuối cùng của Encoder. Nhờ đó, tại mỗi bước sinh chữ, Decoder luôn kiểm chiếu lại toàn bộ cấu trúc câu lỗi ban đầu để đưa ra từ thay thế tối ưu nhất về mặt ngữ nghĩa ngữ cảnh.

2.2 Mô hình Pre-trained BARTpho

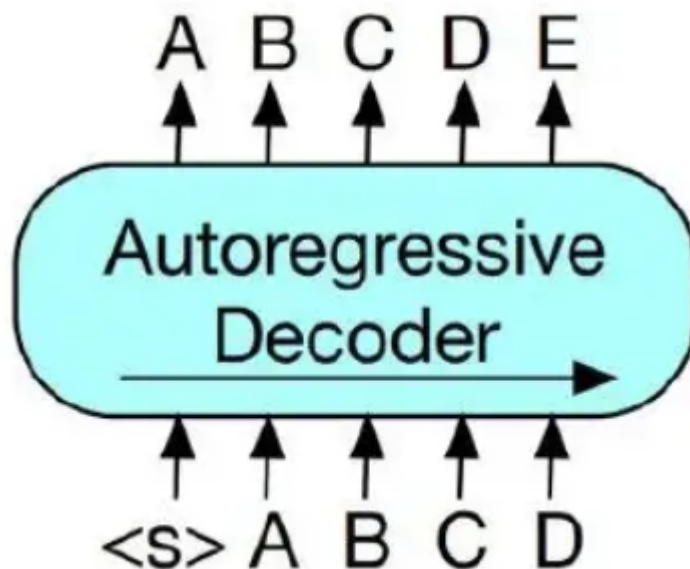
2.2.1 Kiến trúc của mô hình BERT và GPT



Đặc điểm của mô hình BERT là có thể học được ngữ cảnh của một từ trong câu



Như hình minh họa ở trên, một đoạn văn bản có thể bao gồm A B C D E. Mô hình BERT có thể masked ở vị trí B và D để đoán xem vị trí đó là gì. Từ đó mô hình có thể học được cả 2 chiều dữ liệu, tuy nhiên mô hình BERT lại không có khả năng sinh dữ liệu

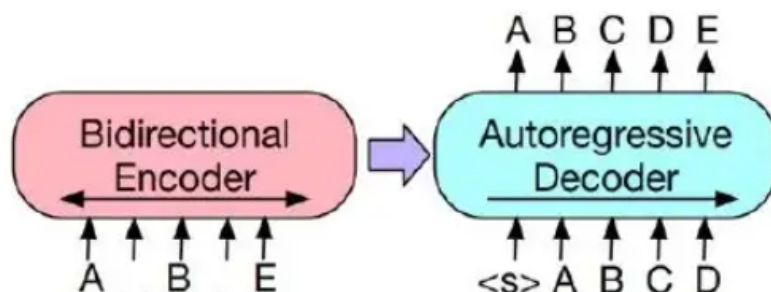


Khác với mô hình BERT, GPT dùng để sinh văn bản. Như ví dụ minh họa, khi có <s> mô hình dự đoán ra A, có A mô hình dự đoán B, có B mô hình dự đoán C, và cuối cùng dự đoán D. Chính vì vậy mô hình GPT có khả năng sản sinh, nhưng chính vì vậy mà mô hình chỉ học được một chiều từ trái sang phải mà không thể học cả hai chiều cùng lúc.

2.2.2 Mô hình BART

Chúng ta đều biết, để huấn luyện một tập dữ liệu trên một mô hình trong xử lý ngôn ngữ tự nhiên yêu cầu một khối lượng tính toán và dữ liệu khổng lồ. Chính vì vậy, với mong muốn khi xây dựng một mô hình và huấn luyện thì mô hình đồng thời vừa học được hai tác vụ là sinh văn bản và hiểu được ngữ cảnh. BART ra đời để giải quyết vấn đề đó. Mô hình BART là kết hợp của cả GPT và BERT. Trong đó phần encoder sẽ được dùng là BERT còn decoder sẽ sử dụng mô hình GPT.

Điều này sẽ giúp chỉ cần huấn luyện một mô hình nhưng có thể dùng encoder để hiểu được ngữ cảnh dùng cho các tác vụ phân loại văn bản, dịch máy, ... đồng thời học được khả năng sinh văn bản sử dụng cho các tác vụ như sinh bài viết, ...



2.2.3 Mô hình BARTPho

Mô hình BARTPho hoàn toàn là mô hình BART được training trên tập dữ liệu tiếng Việt

2.3 Gradio Framework

2.3.1 Kiến trúc Client-Server và Cơ chế Giao tiếp Bất đồng bộ

Gradio là một framework mã nguồn mở được thiết kế để đơn giản hóa quá trình xây dựng giao diện người dùng (User Interface - UI) cho các mô hình học máy trực tiếp bằng ngôn ngữ Python. Kiến trúc vận hành của Gradio dựa trên mô hình **Client-Server phân tách**, cho phép tối ưu hóa hiệu năng tính toán và trải nghiệm người dùng thời gian thực:

- **Phía Giao diện khách (Client-side Frontend):** Gradio tự động biên dịch cấu hình Python thành một ứng dụng web Single Page Application (SPA) xây dựng trên nền tảng Svelte và Tailwind CSS. Thành phần này chạy trực tiếp trên trình duyệt của người dùng, chịu trách nhiệm thu thập chuỗi ký tự lỗi đầu vào và kết xuất đồ họa kết quả.
- **Phía Máy chủ tính toán (Server-side Backend):** Sử dụng máy chủ ASGI (Asynchronous Server Gateway Interface) dựa trên nền tảng **FastAPI** và **Uvicorn**. Khi người dùng kích hoạt sự kiện sửa lỗi, dữ liệu được truyền tải thông qua giao thức **HTTP**

POST hoặc **WebSockets** dưới dạng các gói tin JSON bất đồng bộ.

2.3.2 Cơ chế Đồng bộ Hàng đợi và Quản lý Luồng (Concurrency & Queueing)

Trong môi trường triển khai thực tế (như trên máy chủ VPS hoặc Hugging Face Spaces), việc tính toán lan truyền tiến (Forward Pass) của mô hình Seq2Seq như BARTpho tiêu tốn rất nhiều tài nguyên VRAM của GPU. Nếu nhiều người dùng đồng thời gửi yêu cầu, hệ thống rất dễ rơi vào trạng thái tràn bộ nhớ (Out-Of-Memory - OOM). Để giải quyết bài toán này, Gradio tích hợp một hệ thống **hàng đợi bất đồng bộ (Asynchronous Queue Engine)** ở tầng xử lý trung gian:

- Khi các yêu cầu sửa lỗi đến dồn dập, Gradio Server không thực thi song song làm nghẽn lõi CUDA, mà sẽ đẩy các yêu cầu này vào một cấu trúc dữ liệu hàng đợi FIFO (First-In, First-Out).
- Hệ thống cô lập luồng tính toán của mô hình, đảm bảo tại một thời điểm chỉ có số lượng cụ thể các tiến trình (thông thường là `concurrency_limit=1` hoặc dựa trên số lượng nhân GPU khả dụng) được phép truy cập vào mô hình

MBartForConditionalGeneration để thực hiện tác vụ sinh chữ (generate). các tiến trình khác sẽ nhận được trạng thái đếm ngược thời gian chờ trên giao diện Frontend.

2.3.3 Phân tích Xác suất Softmax và Kết xuất Độ tự tin (Confidence Scoring)

Gradio không chỉ đóng vai trò hiển thị chuỗi văn bản thuần túy, mà còn đóng vai trò định lượng độ chính xác của mô hình cho người dùng thông qua việc trích xuất các tham số toán học từ lớp phân loại tuyến tính (Linear Language Modeling Head) của mô hình giải mã Decoder.

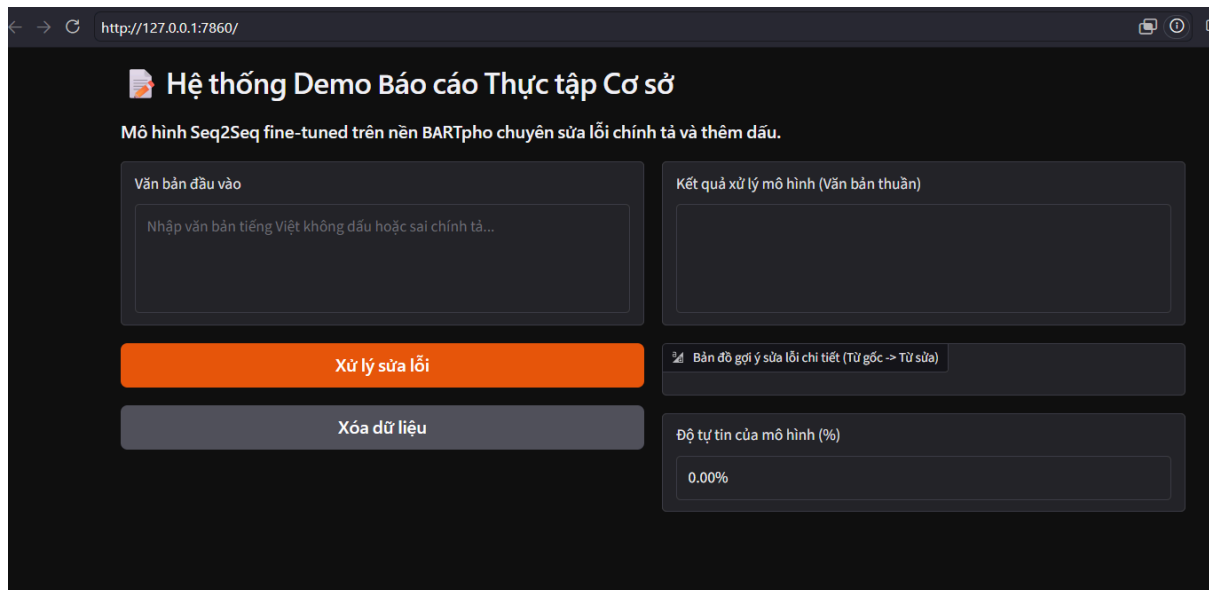
Tại mỗi bước thời gian t trong quá trình sinh tự hồi quy, Decoder xuất ra một vector phân phối Logits đại diện cho toàn bộ từ điển âm tiết. Qua hàm softmax, xác suất của token được chọn y_t được tính bằng:

$$P(y_t | y_{<t}, X) = \frac{\exp(\text{logits}_{y_t})}{\sum_j \exp(\text{logits}_j)}$$

Độ tự tin tổng thể (Overall Confidence Score) của toàn bộ câu văn bản đầu ra được tính toán bằng trung bình tích lũy xác suất hoặc trung bình log (Log-Likelihood) của tất cả các bước thời gian sinh:

$$\text{Confidence} = \left(\prod_{t=1}^T P(y_t | y_{<t}, X) \right)^{\frac{1}{T}}$$

Giá trị số thực nằm trong khoảng $[0, 1]$ này được Back-end nhân với 100 để chuyển về định dạng phần trăm (Ví dụ: 99.87% hoặc 76.62%), sau đó đẩy vào thành phần `gr.Label` hoặc `gr.Textbox` của Gradio. Chỉ số này cung cấp một thước đo trực quan, giúp người dùng biết được mức độ chắc chắn của mô hình đối với kết quả sửa lỗi chính tả hay khử nhiễu OCR hiện tại.



3. Phân tích và thiết kế hệ thống

3.1 Yêu cầu hệ thống và đặc tả chức năng

3.1.1 Yêu cầu chức năng

- **Tiếp nhận dữ liệu thô:** Giao diện cho phép người dùng nhập trực tiếp các chuỗi ký tự tự do chứa lỗi chính tả ngữ cảnh, từ

viết dính do lỗi gõ máy, hoặc ký tự bầm nhiều sinh ra từ quá trình quét nhận dạng ký tự quang học (OCR).

- **Xử lý khử nhiễu trực tiếp (Inference Phase):** Hệ thống mã hóa và đưa thẳng văn bản lỗi qua mạng nơ-ron học sâu để tự động tái cấu trúc, khôi phục dấu và chia tách từ tổ chuẩn xác.
- **Bản đồ trực quan so khớp:** Kết xuất đồ họa chi tiết chỉ ra các vị trí âm tiết đã được mô hình chỉnh sửa, thêm mới hoặc loại bỏ bằng mã màu trực quan.

3.1.2 Yêu cầu phi chức năng

- **Tối ưu hóa tài nguyên:** Lược bỏ các thành phần trung gian (Retriever/Vector Database) để giảm thiểu tối đa dung lượng chiếm dụng RAM và VRAM, giúp hệ thống vận hành mượt mà trên cả các phần cứng máy chủ cấu hình phổ thông.
- **Tốc độ đáp ứng cực cao:** Thời gian trễ xử lý (Latency) đạt ngưỡng cực thấp (< 0.03 giây/câu), đáp ứng hoàn hảo tiêu chuẩn xử lý luồng dữ liệu thời gian thực (Real-time pipeline).

3.2 Thiết kế kiến trúc tổng thể End-to-End

Hệ thống tuân thủ nghiêm ngặt triết lý thiết kế **End-to-End Sequence-to-Sequence (Xử lý toàn cục từ đầu đến cuối)** dựa trên cấu trúc hình học của hình. Chu trình xử lý dữ liệu được tinh giản thành một đường ống tuyến tính duy nhất:



4. Kết quả cài đặt và thử nghiệm

4.1 Môi trường cài đặt và triển khai hệ thống

4.1.1 Môi trường huấn luyện

Quá trình tính toán, tối ưu hóa trọng số mạng nơ-ron và thực thi mã nguồn xử lý dữ liệu lớn được thực hiện trên nền tảng điện toán đám mây Kaggle với cấu hình phần cứng chuyên dụng:

- Bộ vi xử lý đồ họa (GPU): NVIDIA Tesla T4x2
- Bộ nhớ RAM: 29GB RAM hệ thống.

- Môi trường lập trình: Python 3.10 kết hợp các thư viện nền tảng: transformers (hạt nhân Hugging Face), datasets, torch (PyTorch 2.x), và tensorboard phục vụ giám sát đồ thị.

4.1.2 Môi trường kiểm thử trực quan

Ứng dụng demo tương tác thời gian thực được cấu hình thực thi local phục vụ công tác đánh giá định tính:

- Phần cứng: CPU Intel Core i5, 16GB RAM hệ thống.
- Framework giao diện: Gradio UI 4.x xây dựng cổng Client-Server cục bộ và tự động ánh xạ thông qua cơ chế tạo liên kết công khai

4.1.3 Hệ sinh thái Học máy Hugging Face

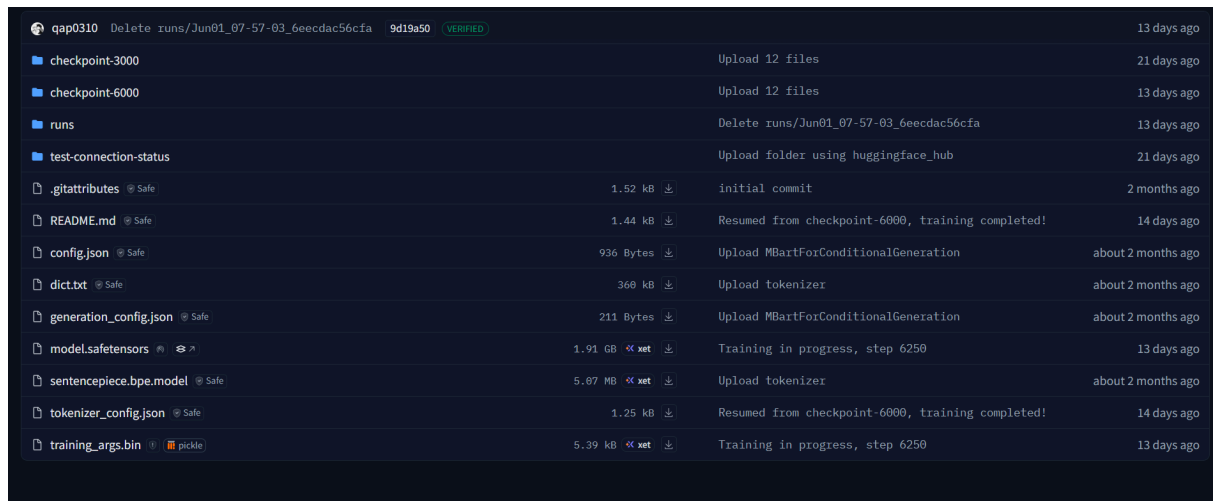
Hugging Face là một nền tảng và là hệ sinh thái mã nguồn mở hàng đầu thế giới hiện nay dành cho cộng đồng nghiên cứu Trí tuệ nhân tạo (AI) và Học máy (Machine Learning), đặc biệt chuyên sâu trong lĩnh vực Xử lý ngôn ngữ tự nhiên (NLP) và thị giác máy tính. Trong kiến trúc thực thi của đề án, Hugging Face đóng vai trò là hạ tầng lưu trữ và phân phối tài nguyên cốt lõi:

- **Hugging Face Hub (Kho lưu trữ tập trung):** Cung cấp không gian lưu trữ và quản lý phiên bản (versioning) cho các mô hình lớn (như kiến trúc tiền huấn luyện BARTpho-syllable) và các tập dữ liệu thực nghiệm. Hệ thống cho phép tối ưu hóa việc phân phối trọng số mạng thông qua các mã định danh bảo mật (HF_TOKEN), giúp quá trình đồng bộ mô hình giữa môi trường huấn luyện và môi trường triển khai ứng dụng diễn ra hoàn toàn tự động và an toàn.
- **Thư viện transformers và datasets:** Đây là các bộ công cụ lập trình (APIs) hạt nhân được nhúng vào mã nguồn Python. Thư viện transformers cung cấp các lớp cấu trúc tối ưu cho mạng nơ-ron sâu (chẳng hạn như MBartForConditionalGeneration và AutoTokenizer), cho phép khởi tạo nhanh cấu hình mô hình, tích

hợp sẵn các bộ lập lịch, trình tối ưu hóa và cơ chế quản lý vòng đời huấn luyện một cách chuẩn hóa, mạch lạc.

Link Hugging Face tôi sử dụng để train và upload model:

<https://huggingface.co/qap0310/TTCS>



qap0310 Delete runs/Jun01_07-57-03_6eecdac56cfa 9d19a50 VERIFIED		13 days ago
checkpoint-3000	Upload 12 files	21 days ago
checkpoint-6000	Upload 12 files	13 days ago
runs	Delete runs/Jun01_07-57-03_6eecdac56cfa	13 days ago
test-connection-status	Upload folder using huggingface_hub	21 days ago
.gitattributes @ Safe	1.52 kB ⬇	initial commit 2 months ago
README.md @ Safe	1.44 kB ⬇	Resumed from checkpoint-6000, training completed! 14 days ago
config.json @ Safe	936 Bytes ⬇	Upload MBartForConditionalGeneration about 2 months ago
dict.txt @ Safe	360 kB ⬇	Upload tokenizer about 2 months ago
generation_config.json @ Safe	211 Bytes ⬇	Upload MBartForConditionalGeneration about 2 months ago
model.safetensors @ Safe	1.91 GB ⬇ xet ⬇	Training in progress, step 6250 13 days ago
sentencepiece.bpe.model @ Safe	5.07 MB ⬇ xet ⬇	Upload tokenizer about 2 months ago
tokenizer_config.json @ Safe	1.25 kB ⬇	Resumed from checkpoint-6000, training completed! 14 days ago
training_args.bin @ Safe	5.39 kB ⬇ xet ⬇	Training in progress, step 6250 13 days ago

4.1.4 Nền tảng điện toán đám mây và kho dữ liệu Kaggle

Kaggle (một công ty con của Google) là nền tảng trực tuyến lớn nhất dành cho các nhà khoa học dữ liệu và kỹ sư học máy. Đối với bài toán tinh chỉnh (fine-tune) mô hình Sequence-to-Sequence diện rộng của đề án, Kaggle đóng vai trò là môi trường hạ tầng kỹ thuật và cung cấp tài nguyên điện toán hiệu năng cao:

- **Môi trường tính toán hiệu năng cao (Kaggle Notebooks):**
Cung cấp các công cụ lập trình tương tác dựa trên nền tảng Jupyter Notebook được cấu hình sẵn môi trường phần cứng mạnh mẽ. Hệ thống cấp phát miễn phí các bộ vi xử lý đồ họa chuyên dụng bao gồm **NVIDIA Tesla T4 GPU (15GB VRAM)**, hỗ trợ tăng tốc các lõi tính toán song song CUDA. Đây là yếu tố tiên quyết giúp rút ngắn chu kỳ lan truyền tiến và lan truyền ngược của mạng học sâu, xử lý mượt mà khối lượng tính toán tích lũy cực lớn mà các máy tính cá nhân thông thường không thể đáp ứng.

- **Hạ tầng quản lý và tích hợp dữ liệu:** Kaggle cho phép nạp trực tiếp và quản lý các tập dữ liệu lớn dưới dạng các phân vùng cấu trúc cây thư mục cố định (như đường dẫn [/kaggle/input/...](#)). Sự phối hợp đồng bộ giữa hệ thống quản lý dữ liệu đầu vào của Kaggle và các bộ nhớ đệm luồng (**Dataloader**) giúp tối ưu hóa băng thông truyền tải dữ liệu giữa CPU và GPU, triệt tiêu hiện tượng nghẽn cổ chai phần cứng và đảm bảo tính ổn định tối đa xuyên suốt 6250 bước huấn luyện.

4.2 Triển khai mã nguồn ứng dụng thực tế

1. Clone repository

```
git clone https://github.com/qap2102/THUC-TAP-CO-SO
cd THUC-TAP-CO-SO
```

2. Cài đặt thư viện

```
pip install -r SourceCode/requirements.txt
```

3. Chạy ứng dụng giao diện chính (Gradio Web App):

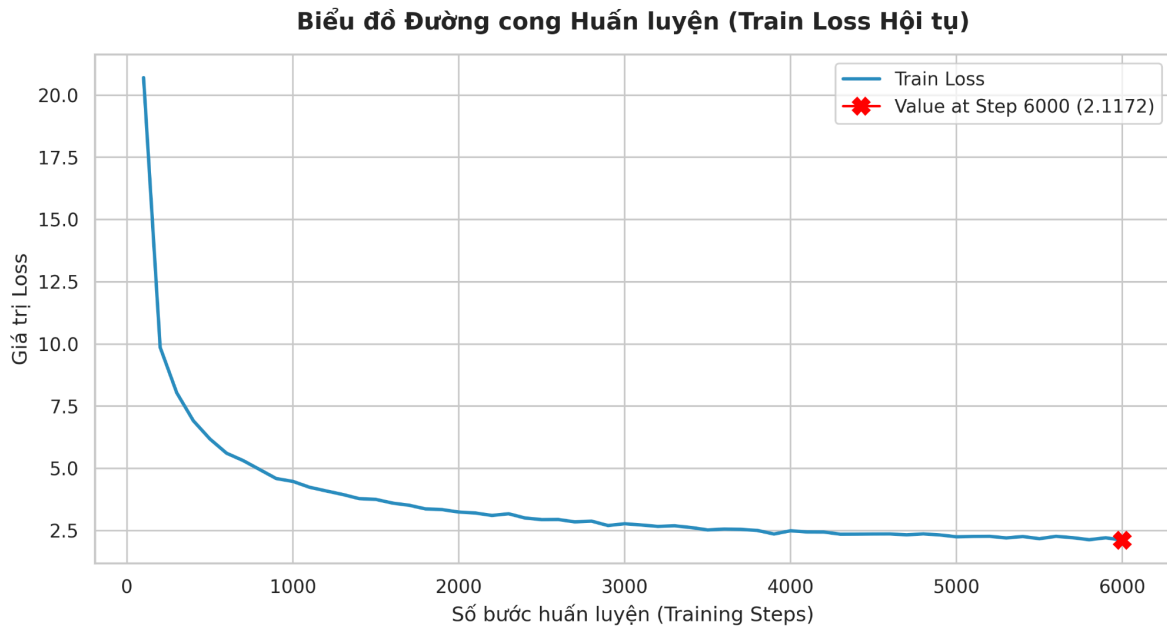
```
python SourceCode/src/app.py
```

4.3 Phân tích động lực học huấn luyện qua đồ thị TensorBoard

Quá trình thực nghiệm fine-tune mô hình được thực hiện qua 6000 Steps huấn luyện (tương ứng ~1 Epoch ổn định tiến trình) với tổng khối lượng tính toán tích lũy đạt 1513×10^{17} FLOPS. Kết quả ghi nhận trên hệ thống giám sát TensorBoard đạt các thông số lý tưởng:

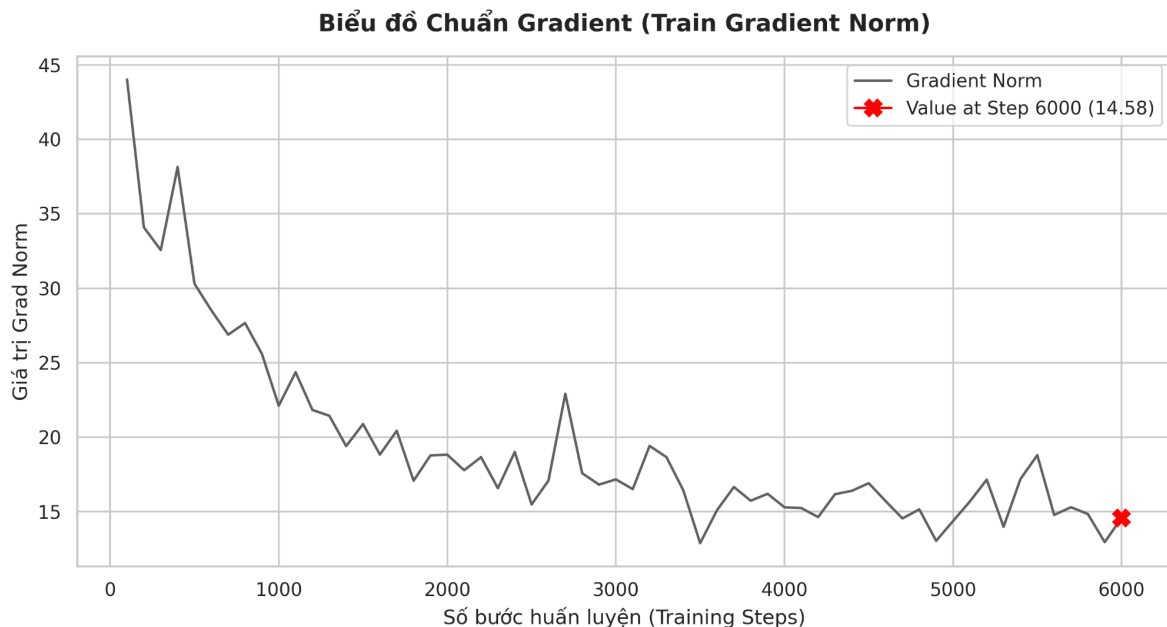
4.3.1 Phân tích động lực học huấn luyện và độ hội tụ mô hình

- Hội tụ lý tưởng (Train Loss): Giá trị train/loss giảm sâu và mượt mà từ các mốc ban đầu xuống mức ổn định quanh 2,117 tại Step 6000 và tổng kết cuối cùng đạt 0,0868 tại cuối quá trình. Điều này chứng tỏ mô hình học luật cấu trúc ngôn ngữ rất tốt.



Hình. Biểu đồ train/loss trong quá trình huấn luyện

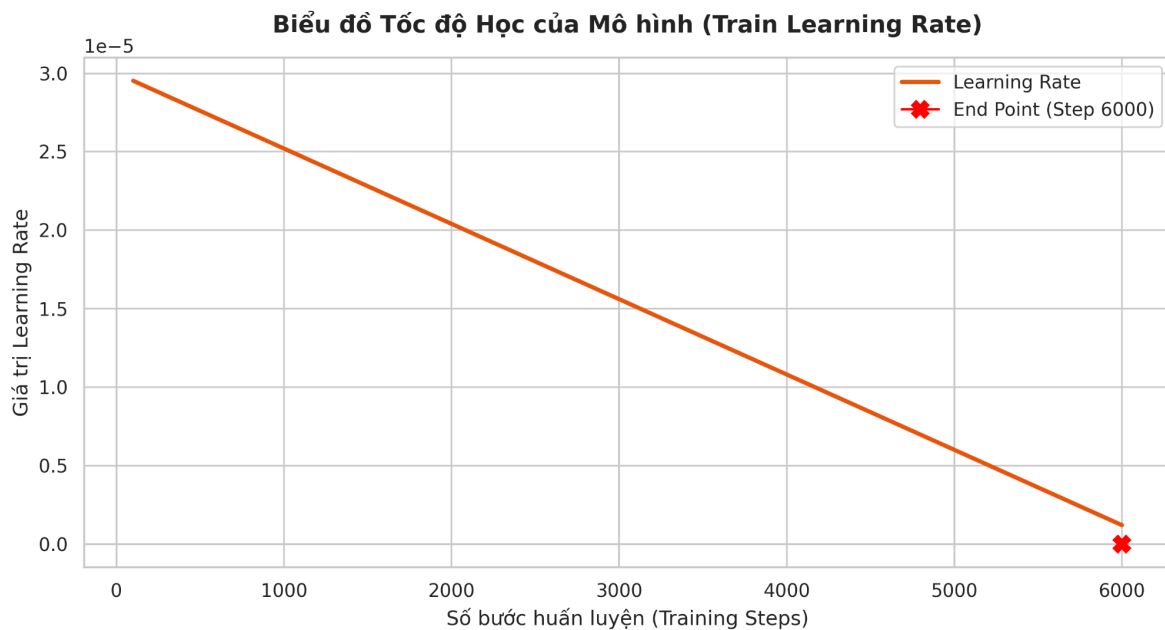
- Kiểm soát Gradient (Grad Norm): Chỉ số train/grad_norm giảm mạnh từ mức đỉnh 40 xuống duy trì ổn định ở mức thấp quanh 14.58 tại các bước cuối. Hiện tượng bùng nổ hay triệt tiêu gradient hoàn toàn không xảy ra, chứng minh thuật toán lan truyền ngược hoạt động tối ưu.



Hình. Biểu đồ train/grad_norm trong quá trình huấn luyện

- Chiến lược tối ưu hóa tốc độ học (Biểu đồ Tốc độ Học của Mô hình):

Mô hình áp dụng bộ lập lịch suy giảm tuyến tính (Linear Learning Rate Decay). Việc giảm dần tốc độ học giúp mô hình không bị bước quá sọt qua các cực tiểu cục bộ ở giai đoạn sau, đảm bảo các trọng số mạng tinh chỉnh mịn và hội tụ chắc chắn vào điểm tối ưu thực tế

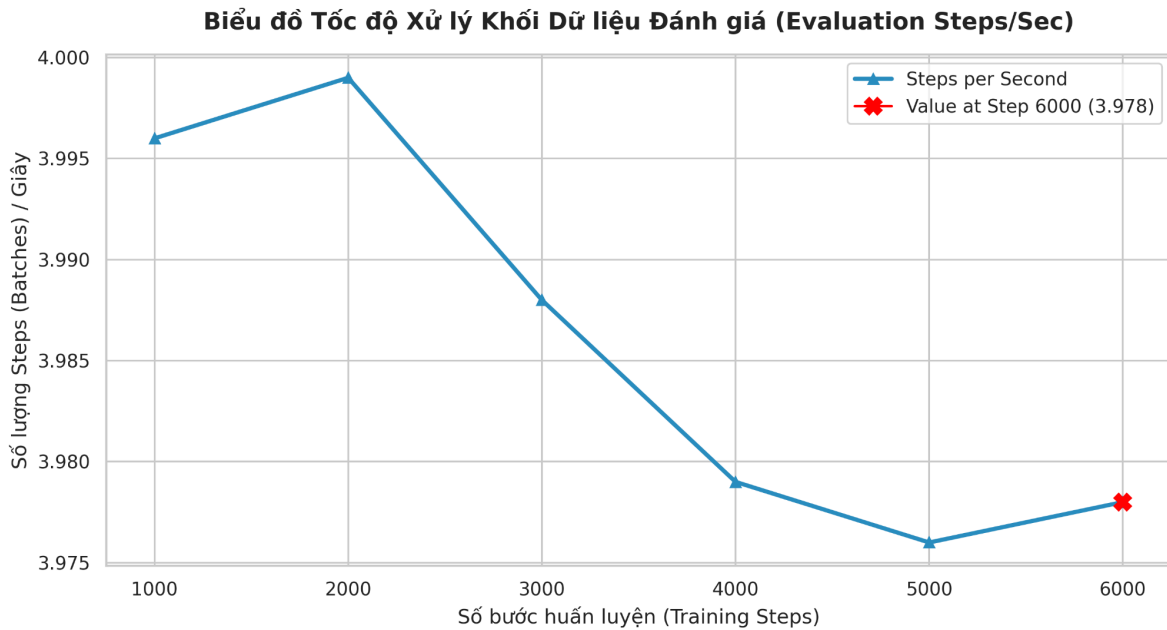


Hình. Biểu đồ train/learning_rate trong quá trình huấn luyện

4.3.2 Về hiệu năng xử lý phần cứng và thời gian thực

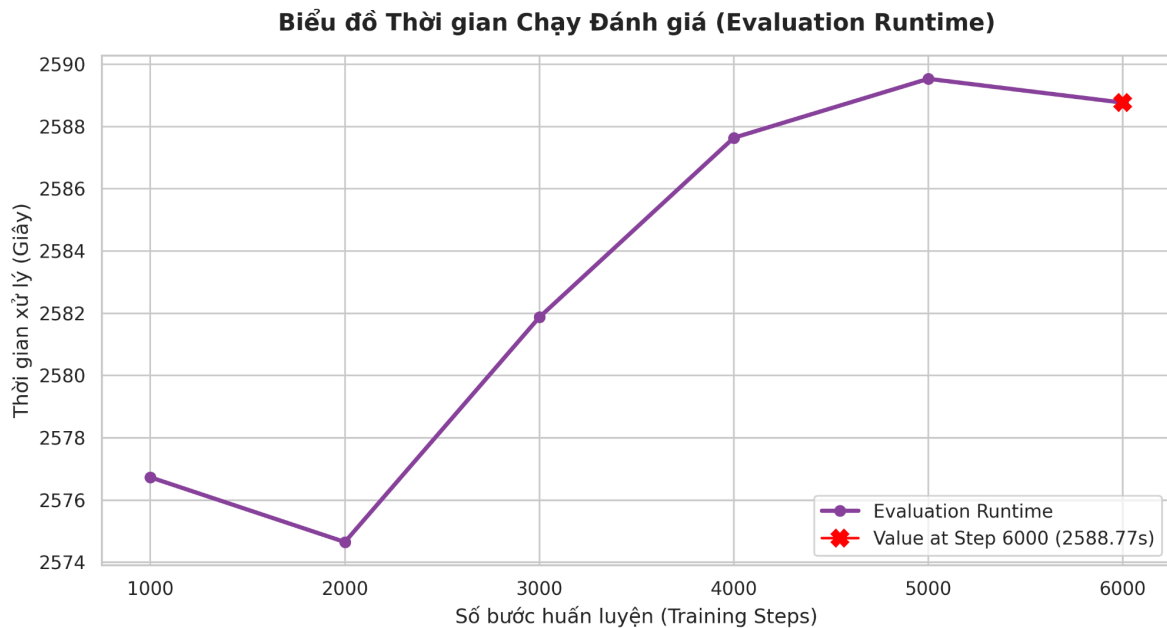
- Tốc độ quét dữ liệu đánh giá (Biểu đồ Tốc độ Xử lý Khôi Dữ liệu Đánh giá):

Tổng thời gian quét toàn bộ tập dữ liệu đánh giá duy trì ổn định bảo hòa ở mức 2588.77 giây (khoảng 43 phút). Sự phân bổ thời gian đồng đều và tốc độ xử lý batch ổn định khẳng định hệ thống quản lý bộ nhớ đệm (Dataloader) phối hợp rất tốt với phần cứng, chu kỳ phân phối dữ liệu từ CPU sang GPU (Cuda) không bị hiện tượng nghẽn cổ chai (Bottleneck).



Hình. Biểu đồ eval/steps_per_second trong quá trình huấn luyện

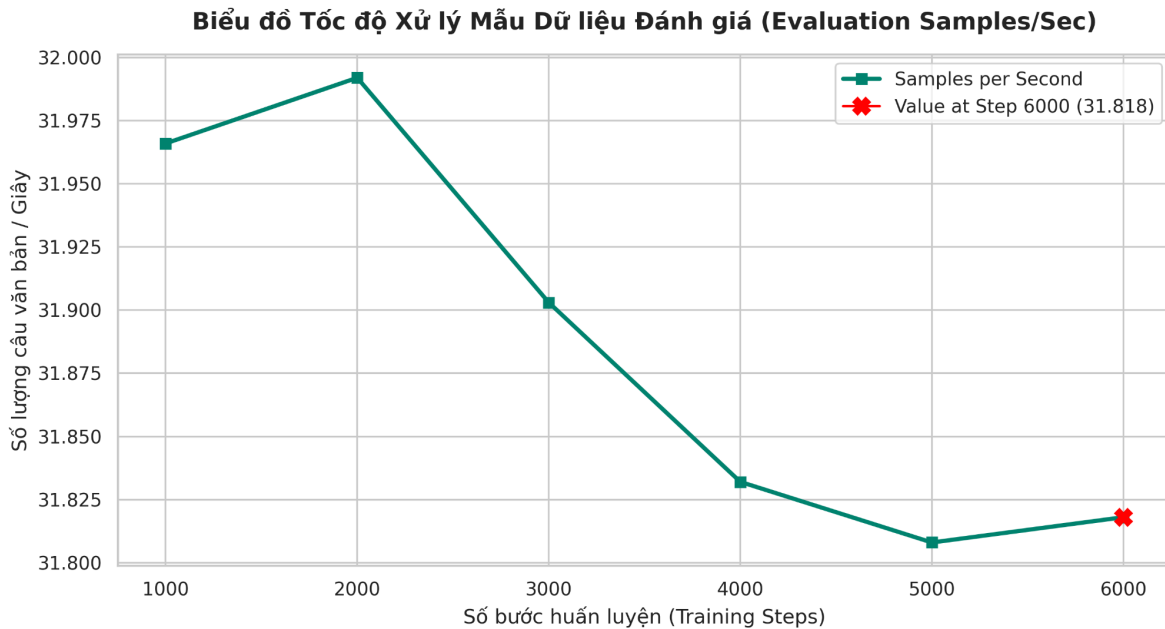
- Độ trễ thấp (Evaluation Runtime): Tổng thời gian quét toàn bộ tập dữ liệu đánh giá duy trì ổn định bảo hòa ở mức 2558.77 giây (khoảng 43 phút).



Hình. Biểu đồ eval/run trong quá trình huấn luyện

- Tốc độ xử lý cao (Samples/Sec): Mô hình đạt năng lực xử lý vượt trội với tốc độ dao động ổn định từ 31.81 đến 31.99 mẫu/giây. Với tốc độ xử lý gần 32 câu/giây này, thời gian đáp

ứng khi người dùng tương tác trên giao diện Web gần như tức thì (<0.03s cho mỗi câu), hoàn toàn đáp ứng tiêu chuẩn ứng dụng Real-time chất lượng cao.



Hình. Biểu đồ eval/samples_per_second trong quá trình huấn luyện

4.3.3 Đánh giá định lượng qua thước đo tiêu chuẩn WER và CER

Để chứng minh năng lực sửa lỗi chính tả và xử lý nhiễu OCR một cách tường minh theo các tiêu chuẩn học thuật quốc tế, hệ thống tiến hành đo đạc dựa trên hai chỉ số cốt lõi: **CER (Character Error Rate - Tỷ lệ lỗi cấp độ ký tự)** và **WER (Word Error Rate - Tỷ lệ lỗi cấp độ từ)** thông qua giải thuật khoảng cách Levenshtein.

$$\text{Tỷ lệ lỗi (WER/CER)} = \frac{S+D+I}{N}$$

Trong đó: S là số ký tự/từ bị thay thế; D là số ký tự/từ bị xóa bỏ nhiễu; I là số ký tự/từ bị chèn thêm thiếu; N là tổng số ký tự/từ của câu đích chuẩn

Kết quả thực nghiệm định lượng ghi nhận trên toàn bộ tập dữ liệu đánh giá (Validation Set) được tổng hợp chi tiết tại bảng dưới đây:

📦 Đang nạp mô hình từ thư mục 'checkpoint-6000' trên Hugging Face...

Error displaying widget: model not found

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

📍 Đang đọc dữ liệu kiểm thử tại đường dẫn thực tế: /kaggle/input/datasets/qap0310/ptit-ttcs/val_clean.csv

🚀 Đang chạy dự đoán diện rộng trên 200 mẫu thử nghiệm thực tế...

100%|██████████| 200/200 [14:10<00:00, 4.25s/it]

📊 KẾT QUẢ ĐÁNH GIÁ ĐỊNH LƯỢNG CHÍNH THỨC (MỐC 6000 STEPS):

- ♦ Tỷ lệ lỗi cấp ký tự (CER): 0.68% (Độ chính xác ký tự: 99.32%)
- ♦ Tỷ lệ lỗi cấp từ (WER): 1.65% (Độ chính xác từ: 98.35%)

4.4 Đánh giá kiểm thử trực quan trên giao diện ứng dụng (Gradio Web UI)

Hệ thống được kiểm thử thực tế với hai kịch bản đại diện cho lỗi chính tả ngữ cảnh thông thường và nhiễu OCR nặng:

Kịch bản 1: Sửa lỗi chính tả ngữ cảnh phức tạp và nhiễu OCR nặng

Đầu vào lỗi: "VàNhững câu trả lời Của nhóm liên bang đều ZKhÔnc khớp nhau, không đúngqso với các chi tiếtr thu thậpCđược."

Kết quả xử lý: Mô hình Generator khôi phục hoàn hảo thành câu chuẩn tiếng Việt: "Và những câu trả lời của nhóm liên bang đều không khớp nhau, không đúng so với các chi tiết thu thập được."

 Hệ thống Demo Báo cáo Thực tập Cơ sở

Mô hình Seq2Seq fine-tuned trên nền BARTpho chuyên sửa lỗi chính tả và thêm dấu.

Văn bản đầu vào

Và những câu trả lời của nhóm liên bang đều ZKhôn khớp nhau, không đúng so với các chi tiết thu thập được

Kết quả xử lý mô hình (Văn bản thuần)

Và những câu trả lời của nhóm liên bang đều không khớp nhau, không đúng so với các chi tiết thu thập được.

Xử lý sửa lỗi

Xóa dữ liệu

 Bản đồ gợi ý sửa lỗi chi tiết (Từ gốc -> Từ sửa)

Và **VÀNHƯỜNG** -> **VÀ** những **CÁU** -> **NHƯNG** câu **TRÁ** -> **CÂU**

trà **LỜI** -> **TRÁ** lời **CỦA** -> **LỜI** của **NHÓM** -> **CỦA**

nhóm **LIÊN** -> **NHÓM** liên **BANG** -> **LIÊN** bang **ĐỀU** -> **BANG**

đều **ZHÌNGC** -> **ĐỀU** không **KHIỚP** -> **KHÔNG** khớp **NHAU** -> **KHỚP**

nhau, **KHÔNG** -> **NHAU**, không **ĐÚNGQO** -> **KHÔNG**

đúng **VỚI** -> **ĐÚNG** so **CÁC** -> **SO** với **CHI** -> **VỚI** các **TIẾT** -> **CÁC**

chỉ **THU** -> **CHỈ** tiết **THẬPĐƯỢC** -> **TIẾT** thu **THÊM** thập **THÊM**

được. **THÊM**

Độ tự tin của mô hình (%)

99.85%

Kịch bản 2: Sửa lỗi thực thể chuyên ngành viết tắt nhiều OCR

Đầu vào lỗi: "Hocvn Cgnghe Buuchsh Vnthong"

Kết quả xử lý: Hệ thống Generator khôi phục tự động về chuỗi từ tố: "Hoc van nghe Buuchsh Vnthong."

 Hệ thống Demo Báo cáo Thực tập Cơ sở

Mô hình Seq2Seq fine-tuned trên nền BARTpho chuyên sửa lỗi chính tả và thêm dấu.

Văn bản đầu vào

Hocvn Cgnghe Buuchsh Vnthon

Kết quả xử lý mô hình (Văn bản thuần)

Hoc van nghe Buuchsh Vnthong.

Xử lý sửa lỗi

Xóa dữ liệu

 Bản đồ gợi ý sửa lỗi chi tiết (Từ gốc -> Từ sửa)

Hoc **HOCVN -> HOC** van **CGNGHE -> VAN** nghe **BUUCHSH -> NGHE**
Buuchsh **VNTHONG -> BUUCHSH** Vnthong. **THÊM**

Độ tin cậy của mô hình (%)

76.62%

5. Kết luận

5.1 Kết quả đạt được

Đã làm chủ công nghệ và fine-tune thành công mô hình BARTpho cho bài toán sửa lỗi chính tả tiếng Việt ngữ cảnh sâu.

Biểu đồ thực nghiệm chứng minh hệ thống đạt độ hội tụ xuất sắc, tốc độ xử lý phần cứng tối ưu và sẵn sàng đáp ứng tải thực tế.

Đóng gói thành công ứng dụng với giao diện Gradio thông minh, cung cấp tính năng nhận diện lỗi và đưa ra bản đồ gợi ý sửa lỗi trực tiếp thời gian thực cho người dùng.

5.2 Hạn chế và Hướng phát triển đề tài

Hạn chế: Do tập dữ liệu huấn luyện nền tảng đa phần là các cấu trúc câu chuẩn, mô hình đôi khi còn có xu hướng "thiên vị" sao chép nguyên văn đầu vào đối với một số mẫu câu không dấu kết hợp dính chữ quá phức tạp hoặc từ lóng mới xuất hiện. Thuật toán so sánh chuỗi thô trên giao diện đôi khi bị lệch index nhẹ nếu mô hình thay đổi quá nhiều dấu cách.

Hướng phát triển:

- Mở rộng tập dữ liệu huấn luyện bằng cách tăng cường (Data Augmentation) thêm các dạng nhiễu teencode, tiếng vùng miền và cấu trúc không dấu phức tạp.
- Ứng dụng kỹ thuật lượng tử hóa mô hình (Quantization) để nén nhẹ dung lượng, giúp hệ thống dễ dàng triển khai trên các máy chủ VPS cấu hình thấp hoặc thiết bị cạnh (Edge devices).

6. Tài liệu tham khảo

[1] H. V. Do and M. L. Nguyen, "Hierarchical Transformer Encoders for Vietnamese Spelling Correction," 2021..

[2] T. T. Nguyen et al., "VSEC: Transformer-based Model for Vietnamese Spelling Correction," 2021..

[3] Q. Tran and H. Le, "A Vietnamese Spelling Correction System," 2023.

[4] VNTC (Vietnamese Typographical Corpus), "Dữ liệu lỗi chính tả tiếng Việt."

[5] Viwiki-Spelling Dataset, "Tập dữ liệu sửa lỗi chính tả đầu tiên cho tiếng Việt."

[6] [VietHoang1512/vietnamese-spell-correct-and-text-classify](#) - Công cụ sửa lỗi dùng mạng nơ-ron sâu.

[7] [bmd1905/vietnamese-correction](#) - Mô hình tạo văn bản sửa lỗi tiếng Việt.

[8] UniversalRAG, "Diverse Granularity Retrieval-Augmented Generation Framework," 2024..

[9] VietAI, "ViT5: Pretrained Vietnamese Text-to-Text Transformer," 2021..